

Отчет по дипломной работе <https://github.com/Dumowka/rococo>

Оценка: **Хорошо +**

Диплом выполнен с точки зрения кодовой базы **на отличном уровне**, запускается, собирается без необходимости правок кода и/или конфигов, единственный на потоке с рабочем CI Pipeline.

Отмечу **хорошее тестовое покрытие e2e**, тесты успешно проходят как локально, так и в Докере (но есть пара Flaky), при этом **нет тестов в микросервисах**.

Покрыты ключевые пользовательские сценарии на UI (логин/логаут, регистрация, CRUD сущностей через UI), CRUD через REST и gRPC с верификацией результата в БД + базовые негативные проверки.

Есть проблемы с обработкой ошибок

Есть несколько функциональных и архитектурных замечаний

Сильные стороны архитектуры:

1. Корректное разделение на микросервисы по доменам
2. Использование интерфейсов для Spring сервисов
3. Покрытие e2e и pipeline
4. В целом качественная работа с кодом

Основные проблемы:

1. Единый формат ошибок не обеспечен на уровне gateway: нет `@RestControllerAdvice`, часть gRPC-ошибок превращается в неинформативные HTTP 500
2. Непоследовательный маппинг gRPC Status - HTTP (семантика ошибок теряется в большинстве gRPC клиентов gateway)
3. **Риск N+1 в gateway для painting**: при получении списка картин выполняются дополнительные вызовы в сервисы artist и museum (рассмотрим ниже)
4. В Spring Boot модулях unit и integration тесты отсутствуют полностью
5. Не указана локаль для браузера, что приводит к ошибкам в тестах вида:
Actual (1): [Bad credentials]
Expected (1): [Неверные учетные данные пользователя]

Ревью зависимостей

1. Неиспользуемые/лишние зависимости в ряде модулей:
 - rococo-gateway: spring-boot-starter-data-rest, spring-session-core, Lombok
 - rococo-artist/rococo-museum/rococo-painting/rococo-userdata: spring-boot-starter-web при отсутствии контроллеров (сервисы gRPC-only)
2. Рассогласование версий:
 - Spring Boot: rococo-auth = 3.4.4, остальные = 3.5.5

- Jib: rococo-artist = 3.4.5, остальные = 3.4.1

3. Дублирование build-логики (plugins, toolchain, repositories), которое уже задано на root уровне.

Ревью README.md

Недостающие разделы:

- Нет раздела про CI, хотя workflow реализован, обязательно добавь
- Нет раздела про API-документацию (Swagger отсутствует, но я очень советую добавить его в gateway).
- Переменные окружения описаны минимально
- - В README указан /actuator/health для gateway, но в rococo-gateway отсутствует spring-boot-starter-actuator — информация не совпадает с конфигурацией.
- Добавить про запуск тестов локально

Ревью сервисов

rococo-auth

Все сделано в соответствии с Niffler, реализован error.html, что здорово, есть пара замечаний:

- max-lifetime: 1000 в HikariCP - подключения пересоздаются каждую секунду. Рекомендуется 600000 - 1800000 (10-30 минут). Это я что-то отлаживал в Ниффлере и забыл сделать нормально :)
- generate-ddl: true при ddl-auto: none - противоречивые настройк (должно быть false)

rococo-gateway

Хорошие решения:

- Роль API-шлюза с правильным разделением ответственности
- Сервисы имеют интерфейсы отдельно от их реализаций
- Наличие валидации (Bean Validation)

Замечания:

- countryServiceStub создаётся на MUSEUM_CHANNEL_NAME, что логически странно, это выглядит как “случайно совпало” :)
- Нет централизованного обработчика исключений для REST, есть NotFoundException, но нет @RestControllerAdvice

- для списка картин есть риск N+1 (по сетевым вызовам) в PaintingGrpcService:
Список картин берётся одним gRPC вызовом:
`paintingServiceStub.getAllPaintings(...)`
Затем для каждой картины выполняется обогащение:
`artistService.getArtistById(...)`
`museumService.getMuseumById(...)`

Это обязательно надо фиксировать:

- 1) Доработаем proto контракт (тут нам пагинация не нужна, т.к. id мы уже получим из page с картинами, их будет заведомо не больше, чем запрошенная пэйджа с картинами):
`rpc GetArtistsByIds (GetArtistsByIdsRequest) returns (GetArtistsByIdsResponse);`

```
message GetArtistsByIdsRequest {
    repeated string ids = 1;
}
```

```
message GetArtistsByIdsResponse {
    repeated Artist artists = 1;
}
```

Аналогично для музеев

- 2) Реализация в сервисах museum / artist тривиальная:

```
List<MuseumEntity> museums = museumRepository.findAllById(ids);
GetMuseumsByIdsResponse.Builder responseBuilder =
    GetMuseumsByIdsResponse.newBuilder();
for (MuseumEntity museum : museums) {
    responseBuilder.addMuseums(convertToGrpcArtist(museum));
}
responseObserver.onNext(responseBuilder.build());
responseObserver.onCompleted();
```

И в gateway собирать ids и делать всего два запроса:

```
Set<UUID> artistIds = paintings.stream()
    .map(p -> parseUuid(p.getArtistId()))
    .collect(Collectors.toSet());

Set<UUID> museumIds = paintings.stream()
    .map(p -> parseUuid(p.getMuseumId()))
    .collect(Collectors.toSet());

...
artistService.getArtistsByIds(artistIds)
museumService.getMuseumsByIds(museumIds)
...
}
```

rococo-userdata

Замечания:

- gRPC исключения через `RuntimeException` - лучше доменное исключение и единый маппинг через `GrpcAdvice`

rococo-artist

Замечания:

- gRPC исключения через `RuntimeException` - лучше доменное исключение и единый маппинг через `GrpcAdvice`
- Отсутствуют `equals/hashCode` у `entity`.

rococo-museum

Хорошие решения:

- Размещение `Country` и `Museum` в одном `bounded context` - допустимое решение

Замечания:

- gRPC исключения через `RuntimeException` - лучше доменное исключение и единый маппинг через `GrpcAdvice`
- Отсутствие кеширования: справочник стран статичен, но `allCountries()` каждый раз обращается к БД. Для таких справочников хорошо работает аннотация `@Cacheable`
- Имя метода `convertToGrpcArtist` в `MuseumGrpcService` - явная ошибка копипаста
- `updateMuseum` может перезаписать фото пустым значением
- Отсутствуют `equals/hashCode` у `entity`.

Ревью Docker-инфраструктуры

Замечания:

- В Jib порты перепутаны между `museum` и `painting` - `rococo-museum` (порт 9097 в Jib / 9096 в `docker-compose`), `rococo-painting` - наоборот
- Healthcheck ссылается на `actuator`, но `actuator` не подключён в `gateway`
- Сервисы `artist.rococo.dc`, `museum.rococo.dc`, `painting.rococo.dc`, `userdata.rococo.dc` не имеют `healthcheck`
- `localenv.sh` создаёт лишнюю БД `rococo-gateway`

Возможные оптимизации:

- Jib использует JDK вместо JRE, понятно что в ниффлере так же, но можно чуть облегчить сборку eclipse-temurin:21-jdk (~340MB) заменить на eclipse-temurin:21-jre (~200MB).
- Front будет побыстрее собираться если использовать npm ci:


```
FROM node:22.6.0-alpine AS build
ARG NPM
COMMAND

WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --no-audit --no-fund
COPY . ./
RUN npm run ${NPM
COMMAND}
```
- Тесты я бы тоже предложил перевести на базовый image с нужной версией gradle под капотом, чтоб не тратить время на выкачивание gradle wrapper:


```
FROM gradle:8.14-jdk21
WORKDIR /home/gradle/rococo
ENV TZ=Europe/Moscow
COPY ...
CMD gradle test -Dtest.env=docker..
```
- Добавить .dockerignore в корень проекта
- Добавить restart: unless-stopped для Postgre, Kafka, Zookeeper.
- Создать .env файл в корне проекте с с дефолтными значениями переменных PREFIX, PROFILE

Ревью модуля тестов (rococo-e-2-e-tests)

Хорошие решения:

- Качественный код везде: в экстеншенах, PageObjects и в API / DB клиентах
- Верификация через БД - тесты проверяют результат не только через API, но и в БД

Замечания:

- Нет тестов удаления ни для одной сущности (Artist, Museum, Painting) нет тестов на delete.
- Нет тестов на пагинацию по граничным значениям, типа page = -1, size = Integer.MAX_VALUE и т.д.
- Нет тестов на kafka
- Копипаста переменных museums вместо paintings в PaintingRestTest и PaintingGrpcTest
- incorrectTokens() method source можно вынести в общий базовый класс, используется много где
- Дублирование Country-тестов - тесты продублированы в MuseumRestTest и PaintingRestTest. Вынести в отдельный CountryRestTest

- Нет @DisplayName
- REST-тесты: проверка 401 через `assertThrows + assertTrue(contains("401"))`
немного странная проверка через строковое совпадение.